

1) Develop a Python program to implement a Digital Signature scheme using RSA.

Demonstrate message signing and signature verification, and analyze its effectiveness in ensuring integrity and authentication.

Algorithm:

1. **Start** the program.
2. Select two prime numbers p and q .
3. Calculate:
 - o $n = p \times q$
 - o $\phi(n) = (p - 1) \times (q - 1)$
4. Select a public exponent e such that $\gcd(e, \phi(n)) = 1$.
5. Calculate the private key d such that:
 - o $d \times e \bmod \phi(n) = 1$
6. Form the:
 - o **Public key:** (e, n)
 - o **Private key:** (d, n)
7. Enter the original message.
8. Generate the SHA-256 hash of the message.
9. Create the digital signature using the private key.
10. Display the original message and generated signature.
11. For verification, accept the received message.
12. Generate the SHA-256 hash of the received message.
13. Use the public key to recover the hash from the digital signature.
14. Compare the recovered hash with the hash of the received message.
15. If both hashes match, display **PASS – Signature is VALID**.
16. If the hashes do not match, display **FAIL – Signature is INVALID**.
17. Repeat the verification with different test messages to demonstrate tampering detection.
18. **Stop** the program.

Code

```
import hashlib

def generate_keys():

    p = 61

    q = 53

    n = p * q

    phi = (p - 1) * (q - 1)

    e = 17

    d = pow(e, -1, phi)
```

```

    public_key = (e, n)
    private_key = (d, n)
    return public_key, private_key

def hash_message(message):
    hash_value = hashlib.sha256(message.encode()).hexdigest()
    return int(hash_value, 16)

def sign_message(message, private_key):
    d, n = private_key
    message_hash = hash_message(message)
    signature = pow(message_hash, d, n)
    return signature

def verify_signature(message, signature, public_key):
    e, n = public_key
    message_hash = hash_message(message)
    recovered_hash = pow(signature, e, n)
    return recovered_hash == (message_hash % n)

print("=====")
print("    RSA DIGITAL SIGNATURE")
print("=====")
public_key, private_key = generate_keys()
print("\nPublic Key :", public_key)
print("Private Key:", private_key)
original_message = "Hello World"
original_signature = sign_message(
    original_message,
    private_key
)
print("\n=====")

```

```

print("ORIGINAL MESSAGE")

print("=====")

print("Message :", original_message)

print("Signature :", original_signature)

print("\nOriginal message signature generated.")

print("\n=====")

print("SIGNATURE VERIFICATION")

print("=====")

user_message = input(
    "\nEnter message to verify: "
)

print("\nChecking signature...")

result = verify_signature(
    user_message,
    original_signature,
    public_key
)

if result:
    print("\n-----")
    print("RESULT: PASS")
    print("-----")
    print("Signature is VALID.")
    print("Message is authentic.")
    print("Message integrity is maintained.")
else:
    print("\n-----")
    print("RESULT: FAIL")

```

```

print("-----")
print("Signature is INVALID.")
print("Message has been modified or is not authentic.")
print("\n\n=====")
print("    AUTOMATIC TEST CASES")
print("=====")

```

```

test_cases = [
    ("Hello World", True),
    ("Hello World!", False),
    ("hello world", False),
    ("Hello", False),
    ("RSA Digital Signature", False),
    ("123456", False),
    ("Test@123", False),
    ("Hello World ", False),
    ("HELLO WORLD", False),
    ("Cyber Security", False)
]

```

```

passed = 0

```

```

failed = 0

```

```

for i, (test_message, expected) in enumerate(test_cases, 1):

```

```

    result = verify_signature(
        test_message,
        original_signature,

```

```
    public_key
)
```

```
if result == expected:
```

```
    status = "PASS"
```

```
    passed += 1
```

```
else:
```

```
    status = "FAIL"
```

```
    failed += 1
```

```
print(f"\nTest Case {i}")
```

```
print("Input   :", repr(test_message))
```

```
print("Expected :", "VALID" if expected else "INVALID")
```

```
print("Actual   :", "VALID" if result else "INVALID")
```

```
print("Result   :", status)
```

```
print("\n=====")
```

```
print("      TEST SUMMARY")
```

```
print("=====")
```

```
print("Total Tests :", len(test_cases))
```

```
print("Passed   :", passed)
```

```
print("Failed   :", failed)
```

```
if failed == 0:
```

```
    print("\nALL TEST CASES PASSED")
```

```
else:
```

```
    print("\nSOME TEST CASES FAILED")
```

Output

```
=====
                RSA DIGITAL SIGNATURE
=====

Public Key : (17, 3233)
Private Key: (2753, 3233)

=====
ORIGINAL MESSAGE|
=====

Message   : Hello World
Signature : 2622

Original message signature generated.

=====
SIGNATURE VERIFICATION
=====

Enter message to verify: HELLO WORLD

Checking signature...
```

```
-----
RESULT: FAIL
-----

Signature is INVALID.
Message has been modified or is not authentic.
```

AUTOMATIC TEST CASES

Test Case 1

```
Input      : 'Hello World'
Expected   : VALID
Actual     : VALID
Result     : PASS
```

Test Case 2

```
Input      : 'Hello World!'
Expected   : INVALID
Actual     : INVALID
Result     : PASS
```

```
Test Case 3
Input      : 'hello world'
Expected   : INVALID
Actual     : INVALID
Result     : PASS

Test Case 4
Input      : 'Hello'
Expected   : INVALID
Actual     : INVALID
Result     : PASS

Test Case 5
Input      : 'RSA Digital Signature'
Expected   : INVALID
Actual     : INVALID
Result     : PASS

Test Case 6
Input      : '123456'
Expected   : INVALID
Actual     : INVALID
Result     : PASS
```

```
Test Case 7
Input      : 'Test@123'
Expected   : INVALID
Actual     : INVALID
Result     : PASS

Test Case 8
Input      : 'Hello World '
Expected   : INVALID
Actual     : INVALID
Result     : PASS

Test Case 9
Input      : 'HELLO WORLD'
Expected   : INVALID
Actual     : INVALID
Result     : PASS

Test Case 10
Input      : 'Cyber Security'
Expected   : INVALID
Actual     : INVALID
Result     : PASS
```

2) Implement MD5 and SHA-256 hashing algorithms using Python's hashlib library.

Compare their outputs for the same input

Algorithm:

1. **Start** the program.
2. Import Python's `hashlib` library.
3. Read a message from the user.
4. Convert the message into bytes using UTF-8 encoding.
5. Apply the **MD5** hashing algorithm using `hashlib.md5()`.
6. Convert the MD5 hash into hexadecimal format using `hexdigest()`.
7. Apply the **SHA-256** hashing algorithm using `hashlib.sha256()`.
8. Convert the SHA-256 hash into hexadecimal format using `hexdigest()`.
9. Display the original message.
10. Display the generated MD5 hash.
11. Display the generated SHA-256 hash.
12. Compare the MD5 and SHA-256 outputs.
13. If the outputs are different, display **PASS – Hash outputs are different**.
14. Check that the MD5 hash contains **32 hexadecimal characters**.
15. Check that the SHA-256 hash contains **64 hexadecimal characters**.
16. Test the same input again and verify that each algorithm produces the same hash.
17. Change the input slightly and verify that the generated hash changes.
18. Display the test-case results.
19. **Stop** the program.

Code

```
import hashlib

def md5_hash(message):
    return hashlib.md5(message.encode("utf-8")).hexdigest()

def sha256_hash(message):
    return hashlib.sha256(message.encode("utf-8")).hexdigest()

print("=" * 60)

print("      MD5 AND SHA-256 HASHING")

print("=" * 60)

message = input("\nEnter a message: ")

md5_result = md5_hash(message)
```



```

sha256_result = sha256_hash(message)

print("\n" + "=" * 60)

print("          HASH RESULTS")

print("=" * 60)

print("\nOriginal Message :", message)

print("\nMD5 Hash:")

print(md5_result)

print("\nSHA-256 Hash:")

print sha256_result

print("\nMD5 Hash Length   :", len(md5_result), "characters")

print("SHA-256 Hash Length :", len sha256_result, "characters")

print("\n" + "=" * 60)

print("          COMPARISON")

print("=" * 60)


if md5_result != sha256_result:

    print("MD5 and SHA-256 outputs are DIFFERENT.")

else:

    print("MD5 and SHA-256 outputs are SAME.")

print("\n\n" + "=" * 60)

print("          TEST CASES")

print("=" * 60)


test_cases = [

    "Hello World",

    "Hello",

    "hello",

    "123456",

```

```
"Test@123",  
"Cyber Security",  
"Digital Signature",  
"MD5",  
"SHA-256",  
"Hello World!"  
]
```

```
passed = 0
```

```
failed = 0
```

```
for i, test_message in enumerate(test_cases, 1):
```

```
    md5_value = md5_hash(test_message)
```

```
    sha256_value = sha256_hash(test_message)
```

```
    md5_length_valid = len(md5_value) == 32
```

```
    sha256_length_valid = len(sha256_value) == 64
```

```
    hashes_different = md5_value != sha256_value
```

```
    if md5_length_valid and sha256_length_valid and hashes_different:
```

```
        result = "PASS"
```

```
        passed += 1
```

```
    else:
```

```
        result = "FAIL"
```

```
        failed += 1
```

```
    print("\nTest Case", i)
```

```
    print("Input          :", repr(test_message))
```

```
    print("MD5           :", md5_value)
```

```
print("SHA-256      :", sha256_value)

print("MD5 Length Valid  :", md5_length_valid)

print("SHA-256 Length Valid:", sha256_length_valid)

print("Outputs Different :", hashes_different)

print("Result      :", result)

print("\n" + "=" * 60)

print("          SAME INPUT TEST")

print("=" * 60)
```

```
same_input = "Hello World"
```

```
md5_first = md5_hash(same_input)

md5_second = md5_hash(same_input)
```

```
sha_first = sha256_hash(same_input)

sha_second = sha256_hash(same_input)
```

```
if md5_first == md5_second:

    print("MD5 Same Input Test   : PASS")

else:

    print("MD5 Same Input Test   : FAIL")
```

```
if sha_first == sha_second:

    print("SHA-256 Same Input Test : PASS")

else:

    print("SHA-256 Same Input Test : FAIL")

print("\n" + "=" * 60)

print("          CHANGED INPUT TEST")
```

```
print("=" * 60)
```

```
original = "Hello World"
```

```
changed = "Hello world"
```

```
md5_original = md5_hash(original)
```

```
md5_changed = md5_hash(changed)
```

```
sha_original = sha256_hash(original)
```

```
sha_changed = sha256_hash(changed)
```

```
if md5_original != md5_changed:
```

```
    print("MD5 Changed Input Test    : PASS")
```

```
else:
```

```
    print("MD5 Changed Input Test    : FAIL")
```

```
if sha_original != sha_changed:
```

```
    print("SHA-256 Changed Input Test : PASS")
```

```
else:
```

```
    print("SHA-256 Changed Input Test : FAIL")
```

```
print("\n" + "=" * 60)
```

```
print("                FINAL SUMMARY")
```

```
print("=" * 60)
```

```
print("Total Test Cases :", len(test_cases))
```

```
print("Passed          :", passed)
```

```
print("Failed          :", failed)
```

```
if failed == 0:
```

```
    print("\nALL TEST CASES PASSED")
else:
    print("\nSOME TEST CASES FAILED")
print("\n" + "=" * 60)
print("          CONCLUSION")
print("=" * 60)
print("""
MD5 generates a 128-bit hash value.
SHA-256 generates a 256-bit hash value.
For the same input, each algorithm produces
the same hash every time.
MD5 and SHA-256 produce different outputs
because they are different hashing algorithms.
SHA-256 is more secure than MD5 and is preferred
for modern security applications.
""")
```

Output

Clear

MD5 AND SHA-256 HASHING

Enter a message: hello world

HASH RESULTS

Original Message : hello world

MD5 Hash:

60c08e2b03b716a176aeb9c2a2fddb79

SHA-256 Hash:

9e1b04359ce9f650852b7f468eb9cbaa8198d8678a7ddce5caaeb123b76439ff

MD5 Hash Length : 32 characters

SHA-256 Hash Length : 64 characters

COMPARISON

MD5 and SHA-256 outputs are DIFFERENT.

TEST CASES

Test Case 1

Input : 'Hello World'

MD5 : b10a8db164e0754105b7a99be72e3fe5

SHA-256 :
a591a6d40bf420404a011733cfb7b190d62c65bf0bcda32b57b277d9ad9f14
6e

MD5 Length Valid : True

SHA-256 Length Valid: True

Outputs Different : True

Result : PASS

Test Case 2

Input : 'Hello'
MD5 : 8b1a9953c4611296a827abf8c47804d7
SHA-256 :
185f8db32271fe25f561a6fc938b2e264306ec304eda518007d17648263819
69
MD5 Length Valid : True
SHA-256 Length Valid: True
Outputs Different : True
Result : PASS

Test Case 3

Input : 'hello'
MD5 : 5d41402abc4b2a76b9719d911017c592
SHA-256 :
2cf24dba5fb0a30e26e83b2ac5b9e29e1b161e5c1fa7425e73043362938b98
24
MD5 Length Valid : True
SHA-256 Length Valid: True
Outputs Different : True
Result : PASS

Test Case 4

Input : '123456'
MD5 : e10adc3949ba59abbe56e057f20f883e
SHA-256 :
8d969eef6ecad3c29a3a629280e686cf0c3f5d5a86aff3ca12020c923adc6c
92
MD5 Length Valid : True
SHA-256 Length Valid: True
Outputs Different : True
Result : PASS

Test Case 5

Input : 'Test@123'
MD5 : f925916e2754e5e03f75dd58a5733251
SHA-256 :
8776f108e247ab1e2b323042c049c266407c81fbad41bde1e8dfc1bb66fd26
7e
MD5 Length Valid : True
SHA-256 Length Valid: True
Outputs Different : True
Result : PASS